

[55] Analyzing the Impact of Cardinality Estimation on Execution Plans in Microsoft SQL Server

요약

본 논문은 Lee, Dutt, Narasayya, Chaudhuri 가 2023 년에 발표한 논문으로, Microsoft SQL Server 를 대상으로 카디널리티 추정이 실행 계획에 미치는 영향을 심층적으로 분석한다. 논문은 이전 일반적 분석 [54]과 달리, 특정 상용 데이터베이스 시스템에 초점을 맞춘다.

Microsoft SQL Server 는 업계에서 가장 널리 사용되는 데이터베이스 시스템 중 하나이며, SQL Server 자체도 고도로 복잡한 쿼리 옵티마이저를 가지고 있다. 이 논문은 SQL Server 의 specific 한 카디널리티 추정 메커니즘과 그 영향을 분석한다.

논문의 핵심 기여는 다음과 같다. 첫째, SQL Server 특화의 카디널리티 추정 오류 패턴을 식별한다. 둘째, 이러한 오류가 실행 계획 선택에 미치는 구체적인 영향을 정량화한다. 셋째, SQL Server 의 성능 튜닝을 위한 실용적인 권장사항을 제시한다.

특히 이 논문은 새로운 통계 기반 카디널리티 추정 모델(SQL Server 2019 에서 도입된 Cardinality Estimation (CE) 모델)과 레거시 모델을 비교하여, 개선의 정도와 한계를 명확히 한다.

상세분석

55.1 SQL Server 의 카디널리티 추정 메커니즘

Microsoft SQL Server 의 카디널리티 추정은 몇 가지 기본 원리에 기반한다:

첫째, 히스토그램 기반 추정이다. SQL Server 는 각 열에 대해 히스토그램을 유지한다. 히스토그램의 각 버킷은 일정 범위의 값들을 나타내고, 각 버킷의 행 수를 저장한다. 범위 쿼리 (WHERE col > 100 AND col < 500)가 주어지면, 해당 범위에 포함된 버킷들의 행 수를 합산하여 카디널리티를 추정한다.

둘째, 독립성 가정이다. 여러 조건이 WHERE 절에 있을 때, 각 조건이 독립적이라고 가정한다. 예를 들어:

WHERE col1 > 100 AND col2 < 50

이 경우, col1 > 100 의 선택도와 col2 < 50 의 선택도를 곱한다.

셋째, 조인의 동등성 가정이다. A JOIN B ON A.id = B.id 에서, A 의 각 행이 B 에서 정확히 하나의 행과 매칭된다고 가정한다. 실제로는 중복 조인(duplicate join)이나 부분 조인(partial join)이 발생할 수 있다.

SQL Server 는 여러 버전의 카디널리티 추정 모델을 제공한다:

- 레거시 CE (Cardinality Estimation): 구 모델로, 단순한 독립성 가정 사용
- 신규 CE (Cardinality Estimation for SQL Server 2014 이상): 더 정교한 모델로, 일부 상관관계 고려

55.2 카디널리티 추정 오류의 영역별 분석

논문은 SQL Server 에서 카디널리티 추정 오류가 심각한 영역들을 식별한다:

첫째, 다중 조건 쿼리이다. WHERE col1 > 100 AND col2 > 200 AND col3 > 300 같은 쿼리에서 여러 조건의 선택도를 곱하면, 오류가 지수적으로 증가한다. 예를 들어, 각 조건의 추정 오류가 2 배씩이면, 3 개 조건의 조합은 8 배 오류가 발생한다.

둘째, 조인 쿼리이다. 여러 테이블 조인에서:

- 조인 순서 결정 단계에서의 오류
- 각 조인 후 결과 카디널리티 추정의 오류 누적
- 특히 조인 선택도(join selectivity) 추정의 어려움

셋째, 특정 데이터 패턴이다:

- 스쿼드 데이터: 특정 값이 과다 대표될 때, 균등 분포 가정이 깨짐
- 클러스터링된 데이터: 데이터가 특정 범위에 밀집되어 있을 때, 히스토그램 기반 추정이 부정확
- 시간 시계열 데이터: 시간에 따라 데이터 분포가 변할 때, 이전 통계가 현재 데이터를 반영하지 못함

55.3 신규 CE vs 레거시 CE 의 비교

논문은 SQL Server 2019 에서 도입된 신규 CE 와 레거시 CE 를 비교한다:

레거시 CE 의 문제점:

- 독립성 가정의 극단적 적용
- 매우 작은 결과 크기의 과소 추정 (0 으로 반올림)
- 매우 큰 결과 크기의 과다 추정
- 조인 선택도의 임의적 상수값 사용

신규 CE 의 개선:

- 기본 독립성 가정은 유지하되, 일부 상관관계 고려
- 결과 크기 0 인 경우의 특별 처리
- 조인 선택도의 더 정교한 추정
- 특정 패턴(예: 범위 쿼리)에 대한 특별 케이스 처리

성능 비교:

- 대부분의 쿼리에서 신규 CE 가 더 정확한 추정 제공
- 평균적으로 추정 오류를 30~40% 감소
- 하지만 특정 패턴의 쿼리에서는 여전히 큰 오류 (10 배 이상)

55.4 실행 계획에 대한 영향

카디널리티 추정의 부정확성이 실행 계획에 구체적으로 어떻게 영향을 미치는지 분석:

첫째, 조인 순서 변경이다. 잘못된 카디널리티 추정으로 인해:

- A JOIN B JOIN C 가 (A JOIN C) JOIN B 로 변경
- (A JOIN B) JOIN C 가 A JOIN (B JOIN C)로 변경
- 조인 순서에 따라 성능이 10 배 이상 차이

둘째, 조인 알고리즘 변경이다:

- Nested Loop Join 에서 Hash Join 으로 변경 (메모리 사용량 극적 증가)
- Hash Join 에서 Merge Join 으로 변경 (정렬 비용 추가)
- 부적절한 알고리즘 선택으로 인한 성능 저하

셋째, 병렬 처리 결정 변경이다:

- 단일 스레드 실행에서 다중 스레드 병렬 실행으로 변경
- 병렬 처리의 오버헤드가 이득을 초과하는 경우 발생
- 메모리 할당량의 부정확한 결정

55.5 성능 튜닝을 위한 실무적 권장사항

논문은 SQL Server 사용자들을 위한 실용적인 권장사항을 제시한다:

첫째, 통계 유지이다:

- 정기적으로 통계 업데이트 (UPDATE STATISTICS)
- 특히 큰 데이터 변화 후에는 즉시 통계 업데이트
- 통계 샘플링 비율 적절하게 설정 (FULLSCAN vs 샘플링)

둘째, 쿼리 작성 방식이다:

- 복잡한 단일 쿼리보다는 단계별 쿼리로 분해
- 조인 순서를 INNER JOIN 구문으로 명시하기보다는, 옵티마이저에게 결정 위임 (옵티마이저가 더 정교함)
- 서브쿼리 대신 CTE(Common Table Expression) 사용

셋째, 힌트(hint) 사용이다:

- 옵티마이저가 부정확한 결정을 반복적으로 하는 경우, 힌트를 사용하여 강제로 계획 지정
- 하지만 힌트는 임시 해결책이므로, 근본 원인(통계 부정확성 등) 해결 권장

넷째, 쿼리 모니터링이다:

- 예상 행 수(Estimated Rows)와 실제 행 수(Actual Rows)를 비교하여 추정 오류 식별
- Actual Rows 가 Estimated Rows 보다 훨씬 크면, 계획 재컴파일 강제 실행 고려

55.6 새로운 통계 모델의 제안

논문은 SQL Server 의 카디널리티 추정을 더욱 개선하기 위한 방향을 제시한다:

첫째, 열 간 상관관계 추정이다:

- 현재의 단일 열 히스토그램을 넘어, 여러 열 간의 상관관계 학습
- 다변량 히스토그램(multivariate histogram)이나 머신러닝 기반 모델 사용

둘째, 적응형 카디널리티 추정이다:

- 실제 쿼리 실행 결과를 기반으로, 옵티마이저가 실시간으로 추정을 조정
- 다음 유사 쿼리를 위해 학습된 정보 활용

셋째, 머신러닝 기반 접근이다:

- 역사적인 쿼리 실행 데이터로부터 카디널리티 추정 모델 학습
- 새로운 쿼리에 대해 학습된 모델을 사용하여 추정

55.7 본 논문과의 관계

이 논문은 특정 상용 데이터베이스 시스템 (SQL Server)에 초점을 맞춘 실증적 분석이다. Exqutor의 맥락에서 보면:

첫째, 실무적 중요성을 강조한다. SQL Server는 업계에서 가장 중요한 데이터베이스 시스템 중 하나이므로, 이 시스템에서 카디널리티 추정의 문제점을 명확히 한 것은 매우 의미 있다.

둘째, 기존 방법의 한계를 보여준다. SQL Server의 신규 CE도 여전히 많은 경우에 10 배 이상의 오류를 보이므로, 더욱 정교한 방법이 필요함을 시사한다.

셋째, 벡터 데이터베이스 시스템으로의 영감을 제공한다. SQL Server 같은 관계형 데이터베이스와 유사하게, 벡터 데이터베이스도 유사성 쿼리의 카디널리티 추정에 어려움을 겪을 수 있다. Exqutor는 이러한 문제를 해결하는 방법을 제시한다.

추가 제기 문제

1. **일반화 가능성:** SQL Server 에 대한 이 분석이 다른 데이터베이스 시스템 (PostgreSQL, MySQL, Oracle 등)에도 유사하게 적용되는가?
2. **통계의 비용:** 정확한 통계를 유지하는 비용이 추정 개선으로부터의 이득을 정당화하는가?
3. **적응형 솔루션:** 실행 중에 추정을 수정하고 계획을 재컴파일하는 비용과 이득 간의 트레이드오프는?
4. **머신러닝의 필요성:** 이 논문의 발견이 머신러닝 기반 카디널리티 추정 (예: Exqutor)의 필요성을 얼마나 강력하게 지지하는가?